NETWORK PATH SIMULATOR USING DIJKSTRA ALGORITHM

Submitted By
Muhammad Saad Bin Khalid
23K-0797

Maaz Nizami
23K-2052

FAST School of Computing
National University of Computer & Emerging Sciences

Contents

1. Motivation

The rapid advancement in computer networks and communication systems has increased the need for efficient routing techniques. In networking, finding the shortest and most optimized path between two nodes is one of the most important tasks. Although routing algorithms are widely taught in universities, students often find it difficult to understand their practical implementation because most learning remains theoretical. The motivation behind this project is to develop a Network Path Simulator that demonstrates shortest path routing in a visual and interactive manner. The project aims to bridge the gap between theoretical learning and practical understanding by allowing users to create networks, assign path costs, and observe how Dijkstra's Algorithm selects the optimal route. This simulator provides educational as well as practical benefits for networking students and researchers by simplifying complex routing concepts through visualization and interaction.

2. Overview

The proposed project is a graphical simulation system called Network Path Simulator (NPS). The system demonstrates how shortest path routing works in communication networks using Dijkstra's Algorithm. The simulator allows users to create network topologies by adding nodes and weighted edges. Users can then select a source and destination node to calculate the shortest route. The result is displayed graphically along with the total path cost. The main purpose of the project is educational. It provides an interactive learning environment for students studying networking, graph theory, routing protocols, and algorithm design.

2.1. Significance of the Project

Routing algorithms are the backbone of modern communication systems. Every network, including the internet, depends on efficient routing mechanisms to transfer data between devices. Understanding these algorithms is therefore extremely important in the field of computer science and networking. This project is significant because it provides a practical implementation of routing concepts that are usually studied theoretically. The simulator allows students to observe how shortest path calculations are performed step by step.

Importance of the Project

- Helps students understand shortest path routing practically.
- Demonstrates the real-world application of Dijkstra's Algorithm.
- Improves visualization of graph traversal techniques.
- Reduces complexity in understanding routing behavior.
- Provides an interactive learning platform for networking courses.
- Enhances programming and algorithmic problem-solving skills.
- Encourages research in network optimization techniques.

The project combines networking concepts, graph theory, and software engineering into a single educational tool. If implemented successfully, the simulator can become a useful academic resource for networking laboratories and algorithm visualization.

2.2. Description of the Project

The Network Path Simulator is designed to simulate a communication network using graph structures. In the simulator:

- Nodes represent routers or network devices.
- Edges represent communication links between devices.
- Weights represent transmission costs or distances.

The user can interact with the simulator to build a network topology dynamically. After constructing the network, the user selects source and destination nodes. The system then applies Dijkstra's shortest path algorithm to calculate the most optimized route.

Main Functionalities

- Add and remove network nodes and communication links.
- Modify link weights dynamically.
- Save and load network topologies to/from JSON files.
- Display network topology graphically.
- Toggle Auto-Dijkstra mode for real-time recalculations.
- Execute Dijkstra's Algorithm manually or automatically.
- Highlight the shortest path visually.
- Simulate packet delivery with animated visualizations.
- Display the total route cost.

The scope of the project focuses on shortest path routing simulation and visualization. Future enhancements may include advanced routing protocols such as RIP or OSPF.

2.3. Background of the Project

Shortest path algorithms have been widely used in networking and optimization systems for decades. Among all shortest path algorithms, Dijkstra's Algorithm is considered one of the most efficient and widely used techniques for non-negative weighted graphs. The algorithm was introduced by the Dutch computer scientist Edsger W. Dijkstra in 1956. Since then, it has been used in many real-world applications such as:

- Internet routing systems
- GPS navigation systems
- Airline route optimization
- Robotics and artificial intelligence
- Transportation and logistics systems
- Communication network analysis

Graphical simulation systems are commonly used in universities to help students visualize algorithms interactively. Existing networking tools often focus on professional simulation and may be too complex for beginners. This project aims to provide a simplified and educationally focused simulator for academic use.

Technologies Related to the Project The project utilizes concepts from:

- Graph Theory
- Data Structures
- Networking
- Algorithm Design

- GUI-based Application Development

Literature and Study Material The following resources were studied during project planning:

- Networking textbooks
- Research articles on shortest path algorithms
- Python GUI documentation
- Graph visualization libraries
- Academic resources on Dijkstra's Algorithm

2.4. Project Category

The proposed project falls under the following category: **Product Based Project** The project is developed as a software application for educational and simulation purposes. It provides a practical implementation of routing algorithms and network visualization.

3. Features/ Scope/ Modules

The project includes multiple modules and features that make it useful and unique.

- **Interactive Network Topology Creation**
  Users can dynamically create nodes and connect them through weighted edges. This allows users to build customized communication networks according to their requirements.
- **Dijkstra Shortest Path Algorithm**
  The simulator implements Dijkstra's shortest path algorithm for route optimization. The algorithm calculates the minimum-cost path between source and destination nodes efficiently.
- **Graphical User Interface(GUI)**
  A user-friendly graphical interface allows easy interaction with the system. Users can visually observe the network topology and routing behavior.
- **Real-Time Path Visualization**
  The calculated shortest route is highlighted graphically, making it easier for users to understand how routing decisions are made.
- **Network Cost Calculation**
  The simulator calculates and displays the total cost or distance of the selected route.
- **Auto-Dijkstra & Dynamic Recalculation**
  An automatic mode allows the simulator to instantly recalculate and update the shortest path routing tables whenever a node is moved, a link weight is edited, or a connection is deleted.
- **Topology Persistence (Save/Load)**
  Users can save their created network layouts and loaded configurations from JSON files. This prevents the loss of complex topologies and allows users to easily resume their work or share network layouts with others.
- **Interactive Packet Simulation**
  Users can visualize data transmission by selecting a source and destination. The system

animates a packet traveling across the network along the computed shortest path, clearly demonstrating the route the data takes, and elegantly drops the packet if a route is unreachable.

- **Educational Learning Tool**
  The system acts as an educational platform for students studying networking and graph algorithms.
- **Error Handling and Validation**
  The software validates user input and prevents invalid operations such as disconnected routes or duplicate node creation.
- **Modular Design**
  The system is divided into independent modules including:

  1. **User Interface Module** (`ui_header.py`, `ui_sidebar.py`, `ui_right_panel.py`)
     Manages interactive controls and user interaction elements
  2. **Topology Management Module** (`topology_manager.py`, `models.py`)
     Handles network node creation, edge management, and topology persistence
  3. **Canvas/Visualization Module** (`canvas_handler.py`)
     Renders network topology graphically and updates visualization in real-time
  4. **Algorithm/Logic Module** (`logic.py`)
     Implements Dijkstra's shortest path algorithm and routing calculations
  5. **Simulation Module** (`simulation_handler.py`)
     Manages packet simulation, animation, and interactive path tracing
  6. **Theme & Styling Module** (`theme.py`)
     Defines color schemes, fonts, and visual themes for consistency

  This modular structure improves maintainability and scalability.

4. Project Planning

The project development process is divided into several phases to ensure systematic implementation.

Phase	Task Description	Duration
Phase 1	Requirement Analysis and Research	Week 1
Phase 2	System Design and Planning	Week 2
Phase 3	GUI Development	Week 3
Phase 4	Graph and Network Module Development	Week 4
Phase 5	Dijkstra Algorithm Integration	Week 5
Phase 6	Testing and Debugging	Week 6
Phase 7	Documentation and Final Review	Week 7

Responsibility Distribution

Team Member	Responsibilities
Maaz Nizami	GUI development, visualization, testing
Saad Bin Khalid	Algorithm implementation, graph management, documentation

5. Project Feasibility

5.1. Technical Feasibility

The project is technically feasible because all required technologies are available and supported by modern systems. Python provides powerful libraries for graph implementation and GUI development.

Technical Risks

- Complexity in graph visualization
- Managing large network topologies
- Ensuring accurate shortest path calculations
- Maintaining GUI responsiveness

These risks can be minimized through modular implementation and regular testing.

5.2. Economic Feasibility

The project is economically feasible because it requires minimal financial investment.

Development Costs

- Open-source software tools
- Existing computer systems
- Free programming libraries

Benefits

- Educational value
- Practical understanding of networking concepts
- Reusable academic learning tool
- Enhancement of technical skills

The benefits of the project greatly exceed its development cost.

5.3. Schedule Feasibility

The project can be completed within the allocated semester timeline because tasks are divided into manageable phases. Potential delays can be controlled through continuous monitoring and proper responsibility distribution among team members.

6. Hardware and Software Requirements

6.1. Hardware Requirements

- Intel Core i3 or higher processor
- Minimum 4 GB RAM
- 500 MB free disk space
- Standard keyboard and mouse
- Monitor or display unit

6.2. Software Requirements

Operating System

- Windows/Linux/macOS

Programming Language

- Python

IDE/ Development Tools

- Visual Studio Code

Libraries and Frameworks

- tkinter
- heapq
- networkx

7. Diagrammatic Representation of the Overall System

The overall architecture of the Network Path Simulator consists of multiple interconnected modules responsible for handling user interaction, graph processing, shortest path calculation, and visualization.

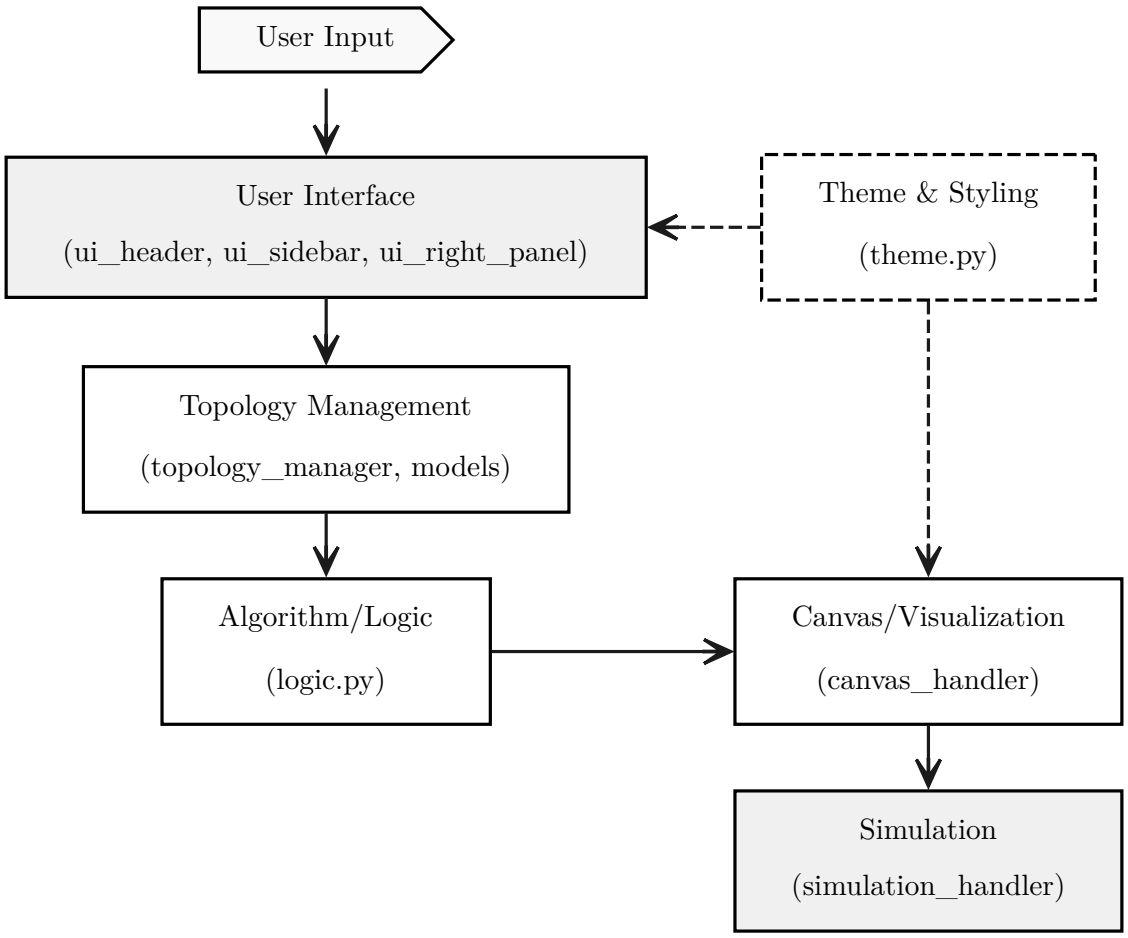

Figure 1: System Architecture and Component Interactivity of the Network Path Simulator.

8. Project Demo and Screenshots

The following screenshots demonstrate the working functionality of the Network Path Simulator, showcasing its various modules and operations.

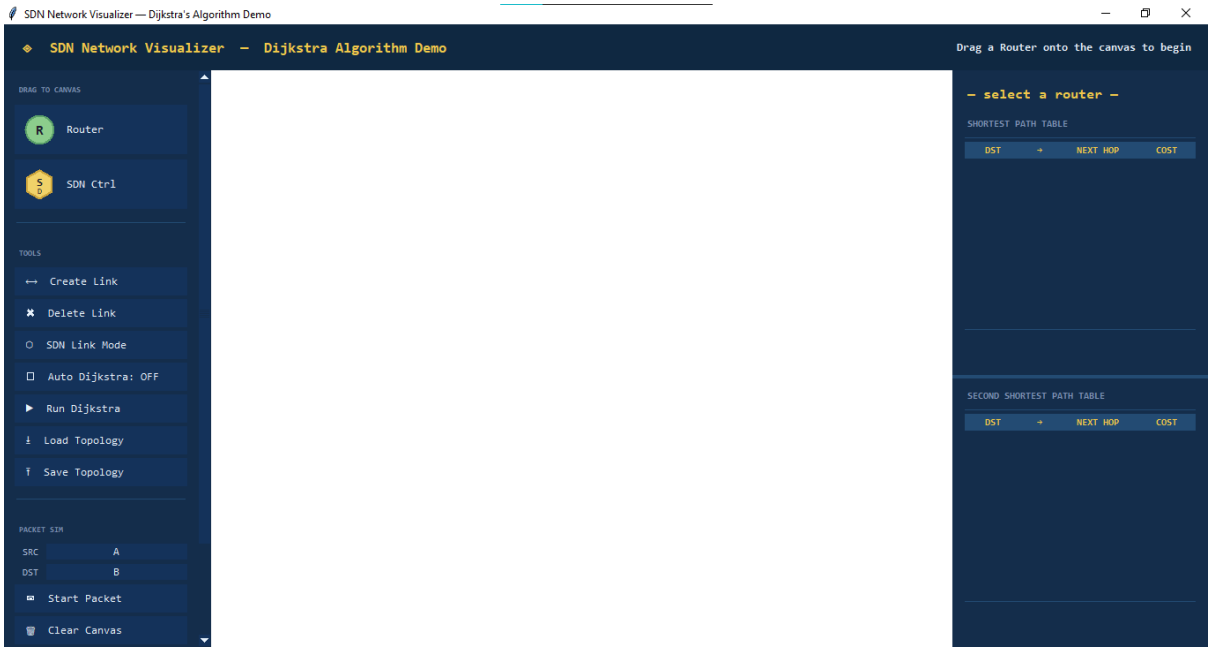

Figure 2: The main user interface with the drawing canvas, left sidebar for function controls and left sidebar for best and second best path tables.

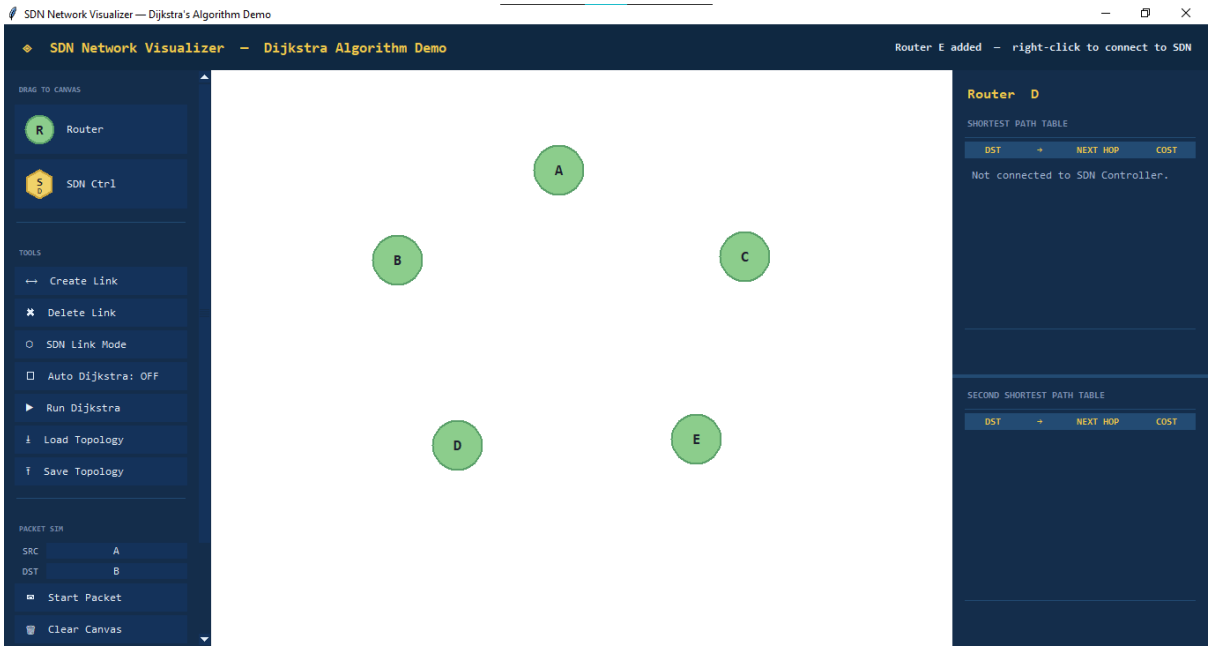

Figure 3: Drag and drop router nodes to create a network topology.

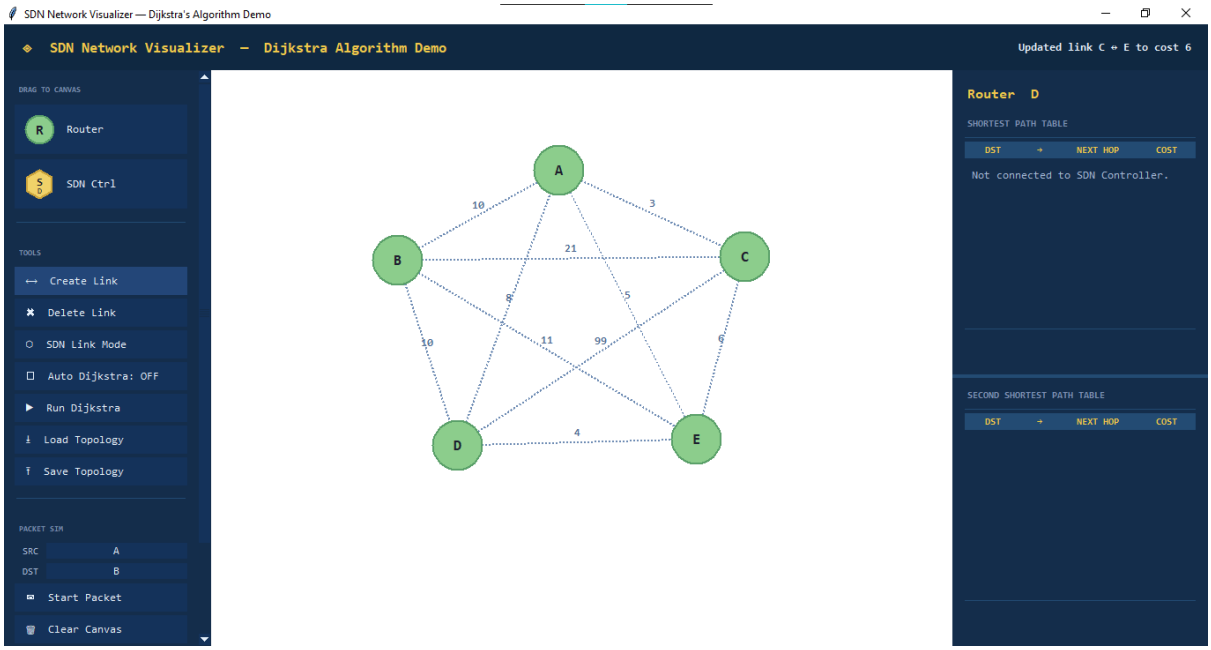

Figure 4: Create links between router nodes to define the network structure by clicking on the create link button and then the source and destination nodes.

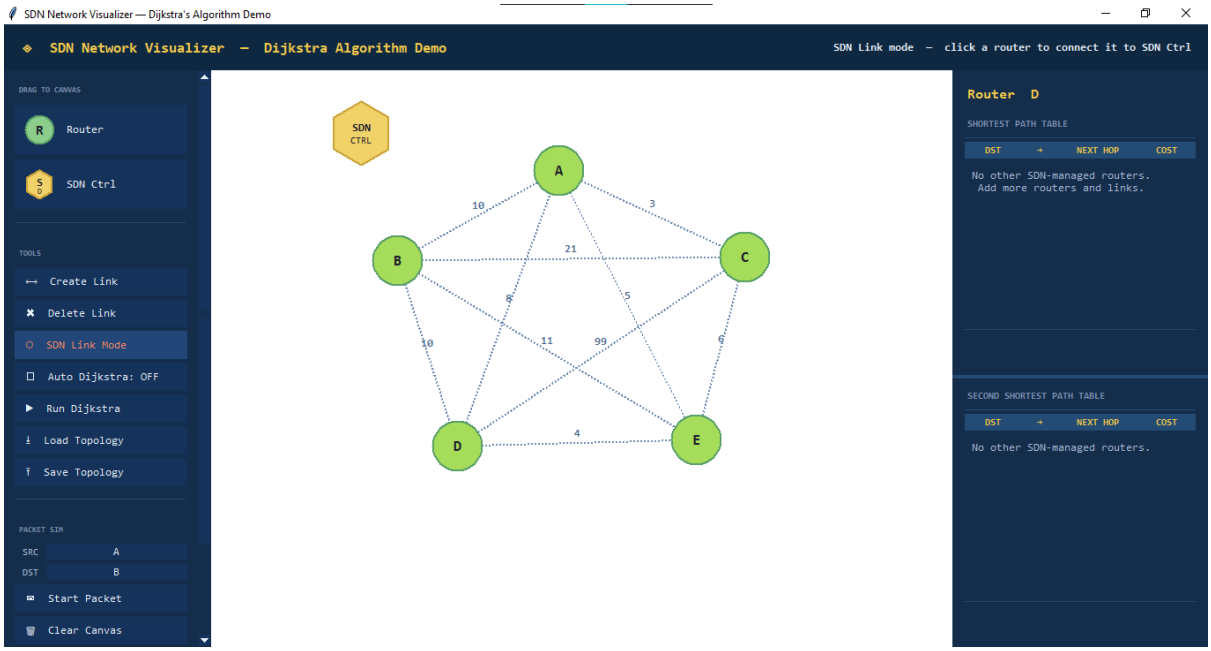

Figure 5: Drag and drop SDN controller and click the SDN link node button to connect the controller to a router node.

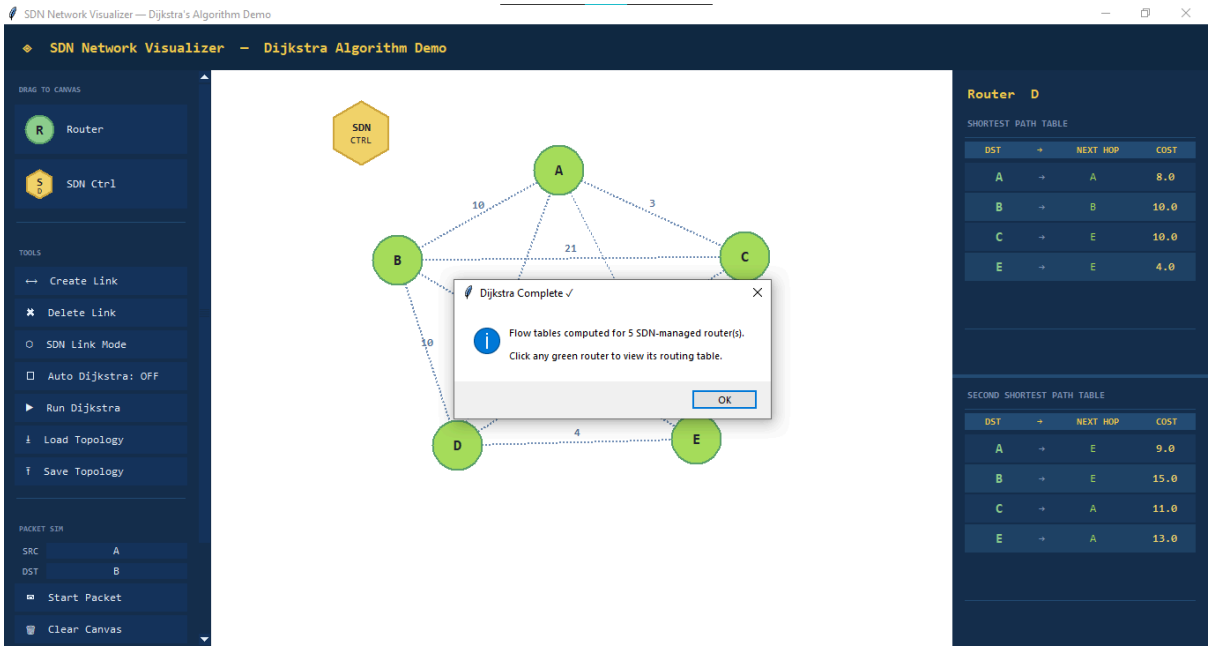

Figure 6: Click the run Dijkstra button to calculate the shortest path between the source and destination nodes. The best path is represented by first table in the left panel and the second best path is represented by the second table.

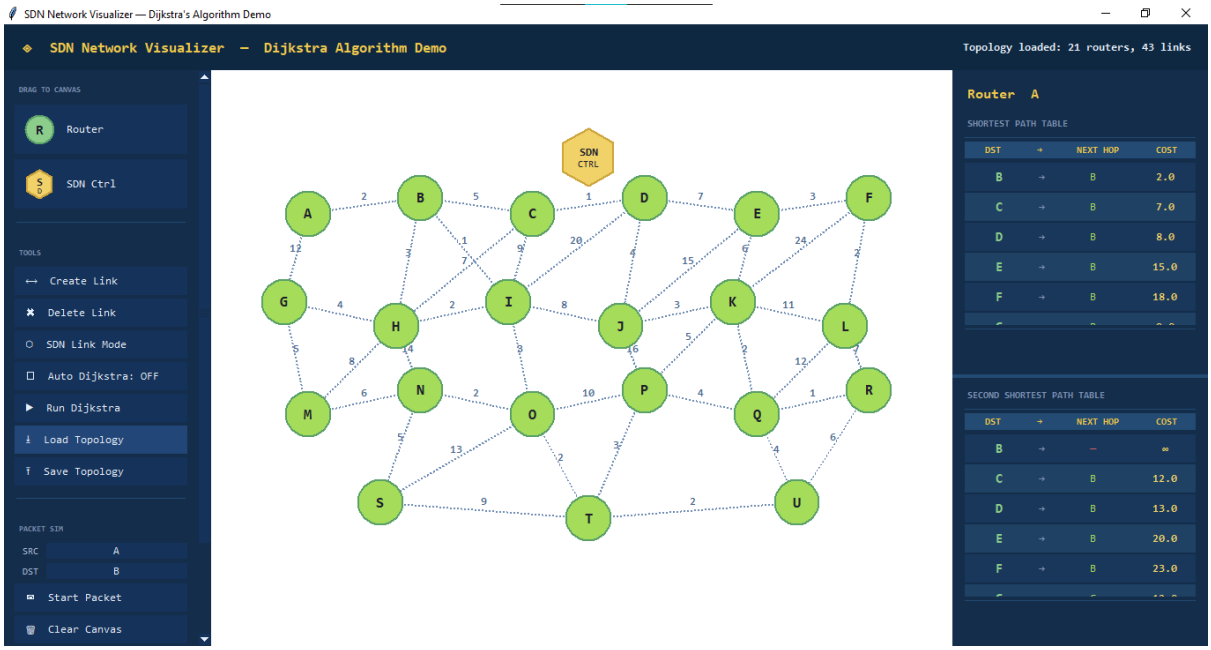

Figure 7: A preconfigured network topology with multiple nodes and links could also be loaded from a JSON file.

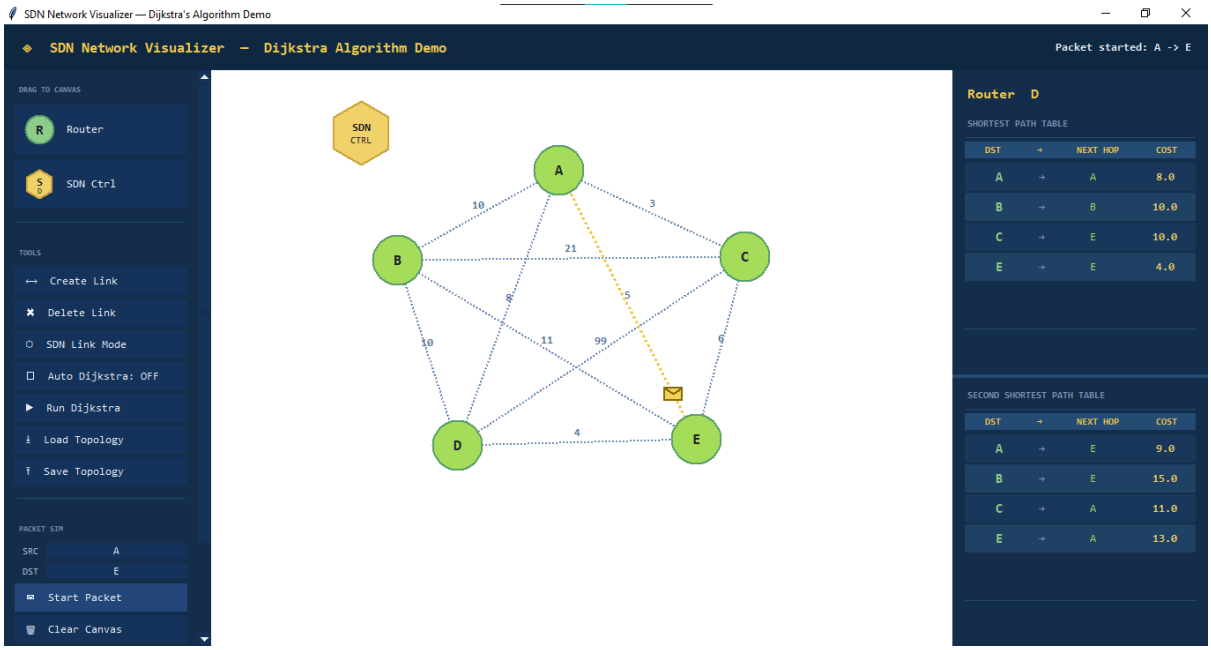

Figure 8: Visualization of the shortest path routing with highlighted nodes and edges representing the best path between the source and destination.

9. References

[1] T.H. Cormen, C.E. Leiserson, R.L. Rivest, and C. Stein, Introduction to Algorithms, MIT Press, Cambridge, 2009.

[2] Andrew S. Tanenbaum and David J. Wetherall, Computer Networks, Pearson Education, New York, 2011.

[3] Edsger W. Dijkstra, "A Note on Two Problems in Connexion with Graphs", Numerische Mathematik, Springer, 1959, pp. 269–271.

[4] Python Software Foundation, "Python Documentation", Available at: https://www.python.org/doc/

[5] NetworkX Developers, "NetworkX Documentation", Available at: https://networkx.org/documentation/

[6] Tkinter Documentation, Available at: https://docs.python.org/3/library/tkinter.html